Interrupt-Driven Arbitration Controller on Zedboard: Project Scope and Workflow

Reconfigurable Computing, AY 2026-27 | Prepared for Devadharsh S

1. Summary

The project builds a configurable hardware arbiter on the Zedboard FPGA (Zynq-7000, logic in the PL only). Two sensors on different buses raise interrupts when data is ready: a PMOD ACL2 accelerometer (SPI) and a PMOD HYGRO temperature/humidity sensor (I2C). A priority-based controller decides which sensor is serviced first, and an aging mechanism stops the lower-priority sensor from starving. The same controller arbitrates the shared outputs, a PMOD OLED display and a UART link to a PC, so each data stream appears intact. The design is verified with a stress testbench, and arbitration latency and fairness are measured and reported.

The intellectual core is the trade-off between responsiveness (high-priority work is served quickly) and fairness (low-priority work is never starved). Everything else is the infrastructure needed to demonstrate that trade-off on real hardware.

2. Problem Statement

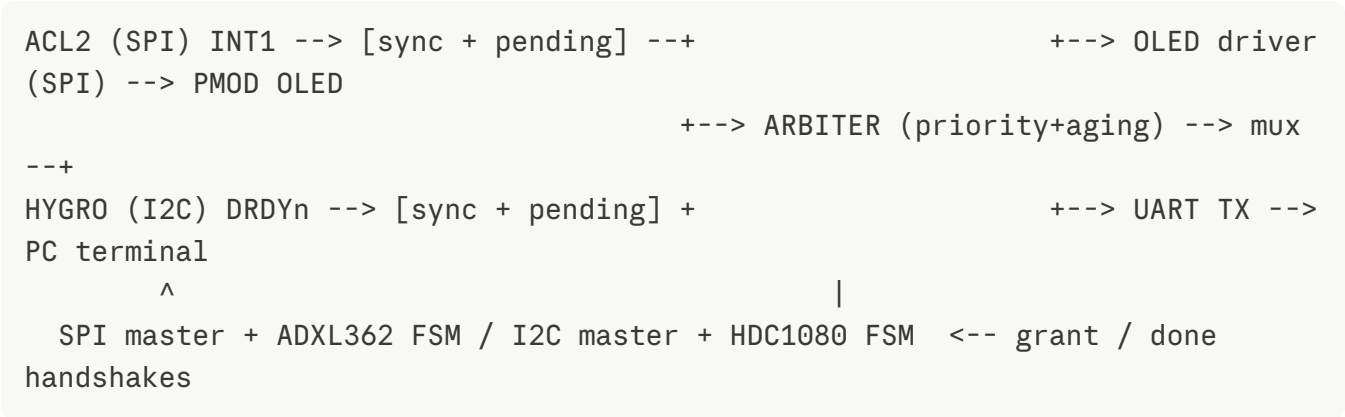
Two data sources produce samples at unpredictable times, but only one OLED and one UART exist. If both sources write at once, the output is garbled. Constantly polling both sensors wastes cycles and adds latency. The system therefore needs to:

- react to sensor events instead of polling (interrupt-driven),
- decide who goes first when both are pending (priority arbitration),
- guarantee that nobody waits forever (aging, bounded waiting),
- give the winner exclusive use of the display and UART until its message is complete (mutual exclusion),
- be configurable (priorities, aging rate, thresholds) without redesigning the hardware.

3. Objectives and Success Criteria

Objective	How success is judged
Event-driven operation	No polling loop; each service starts from a synchronized interrupt
Correct arbitration	Higher effective priority wins; ties resolved deterministically
Starvation freedom	Worst-case wait of the low-priority sensor is bounded under a saturating high-priority load
Clean shared outputs	No interleaved or corrupted messages on OLED or UART
Configurability	Changing priority and aging parameters visibly changes measured results
Verification	Self-checking stress testbench passes with no lost interrupts
Measurement	Arbitration latency (min/avg/max) and fairness metrics reported with and without aging
Hardware demo	Live data on the OLED and in a PC terminal from the Zedboard

4. System Architecture



4.1 Modules

Module	Role
spi_master	Byte-level SPI engine, mode 0, MSB first (already written and loopback-tested)
adxl362_ctrl	Init sequence, interrupt handling, 6-byte XYZ burst read, CS control
i2c_master	START, address, ACK/NACK, data, STOP with open-drain SDA
hdc1080_ctrl	Config write, trigger, DRDYn handling, 4-byte temperature/humidity read
irq_sync_pending (x2)	2-flop synchronizer, polarity handling, edge detect, sticky pending flag
arbiter	Effective-priority calculation, aging counters, grant logic, tie-break
output_mux	Routes the granted source's message to the OLED and UART
msg_formatter	Converts binary sensor values to ASCII/decimal for UART and character codes for OLED
uart_tx	115200 baud, 8N1 transmitter with a small FIFO
oled_driver	SSD1306 init sequence, font ROM, text write over SPI
config_regs	Priority, aging step, aging threshold, enable bits (from switches/buttons or registers)
perf_counters	Timestamps and counters for latency and fairness measurement
top	Wires everything together, clocking, pin ports for the XDC

4.2 Interface convention

Each sensor path presents the arbiter with `req` (pending data), receives `grant`, and returns `done` plus a `data_valid` pulse. A granted transaction is non-preemptive: once it starts, it runs to completion even if priorities change, because SPI and I2C transfers cannot be aborted cleanly mid-frame.

5. Hardware Components

PMOD	Chip	Bus	Direction	Key facts
ACL2	ADXL362	SPI mode 0, up to 8 MHz	Input	3-axis, 12-bit; INT1/INT2 pins; DEVID_AD reads 0xAD; data registers 0x0E-0x13; INTMAP1 0x2A; FILTER_CTL 0x2C; POWER_CTL 0x2D; SOFT_RESET 0x1F
HYGRO	HDC1080	I2C, address 0x40	Input	Temperature and humidity, 14-bit; DRDYn active-low data-ready; registers 0x00 temp, 0x01 humidity, 0x02 config; manufacturer ID 0xFE reads 0x5449 and device ID 0xFF reads 0x1050
OLED	SSD1306 128x32	SPI (write-only) plus D/C, RES, VBATC, VDDC	Output	Needs an init sequence and a font ROM for text
UART	Zedboard USB-UART	Async serial	Output	115200 baud, 8N1, transmit only

Confirm exact pin assignments for each PMOD against the Digilent reference manuals and the Zedboard documentation before writing the XDC, and use PL-connected PMOD headers only.

6. Key Design Decisions

6.1 Interrupt capture

Every asynchronous input goes through a 2-flop synchronizer, then an edge detector (rising for ACL2 INT1, falling for HYGRO DRDYn), then a sticky pending flag cleared only when that service completes. If a new edge arrives during an active read, it must not be lost; decide whether to set pending again or count missed events, and document the choice.

6.2 Arbitration policy

Each requester has a base priority. While a request waits, an aging counter grows, and the effective priority is base priority plus an aging bonus, for example `eff = base + (wait_cycles >> AGE_SHIFT)` saturating at a maximum, or a promotion step each time a threshold is crossed. The arbiter picks the highest effective priority among pending requests when the shared resource becomes free. Ties go to the requester that was served least recently (or a fixed fallback). Aging can be disabled by a configuration bit so the before/after comparison is possible.

6.3 Non-preemption and mutual exclusion

The grant is held from the start of the sensor read through the completed OLED and UART output of that sample. A second source cannot touch either output until `done`. This is what keeps messages from interleaving.

6.4 Buffering

A small FIFO between the sensor read and the output stage absorbs bursts. Track depth, full/empty flags, and overflow so overflow can be reported as a failure in tests.

6.5 Timing style

SPI, I2C and UART run far slower than the 100 MHz clock. Use clock-enable pulses from counters instead of derived clocks, and keep the whole design in one clock domain.

6.6 Making contention happen

On real hardware the ACL2 interrupt rate (about 100 Hz by default) and HDC1080 conversion times (several milliseconds) make simultaneous interrupts rare. To show arbitration and aging convincingly, add (a) a higher ACL2 output data rate, (b) fast re-

triggering of the HYGRO, and (c) a built-in synthetic interrupt generator with configurable rate and burst pattern that can replace the real pins for stress runs. Real sensors prove the interfaces work; the generator proves the arbiter under load.

7. Verification Plan

7.1 Unit level

- spi_master: loopback testbench (done; expected result is all PASS and ALL TESTS PASSED).
- adxl362_ctrl: behavioral ADXL362 slave model with SPI mode 0, 0x0A/0x0B commands, auto-increment, DEVID 0xAD, a known XYZ pattern per sample, and INT1 that clears when data is read.
- i2c_master and hdc1080_ctrl: behavioral HDC1080 slave model with ACK/NACK, pointer register, conversion delay and DRDYn.
- uart_tx: capture and decode the serial stream in the testbench and compare to the expected string.
- oled_driver: check the init byte sequence and the command/data (D/C) split.
- arbiter: directed tests for priority order, ties, and aging promotion.

7.2 System-level stress testbench

- simultaneous interrupts on the same cycle,
- an interrupt arriving mid-transfer and during arbitration,
- back-to-back bursts and random timing with a fixed seed for reproducibility,
- aging counter saturation and FIFO overflow,
- long runs (thousands of events) with self-checking scoreboards that compare every message against the sensor models.

7.3 Coverage targets

Both sensors served, every priority combination exercised, aging triggered at least once, both tie-break paths hit, no lost interrupt, no corrupted message.

8. Measurement Methodology

Metric	Definition	Report as
Arbitration latency	Cycles from pending set (after synchronizer) to grant	min, average, max per sensor
Service latency	Cycles from pending set to last character on the OLED/UART	min, average, max per sensor
Max wait	Longest time any request waited	per sensor
Grant ratio	Grants to sensor A vs sensor B	ratio and counts
Jain's fairness index	(sum x)^2 / (n * sum x^2), with x = per-sensor throughput or share	single number, 1.0 is perfectly fair
Throughput	Completed services per unit time	per sensor and total
Lost events	Interrupts that never produced a service	must be zero

Implement these as counters in `perf_counters` so the same numbers come out of simulation and, optionally, out of the board over UART or through an ILA capture.

Experiments to run

1. Aging OFF, saturating high-priority load: expect the low-priority sensor's max wait to be large or unbounded.
2. Aging ON with the same load: expect a bounded max wait and a better fairness index, at a small latency cost to the high-priority sensor.
3. Sweep aging step and threshold: plot max wait, average latency, and Jain's index against the parameter.
4. Compare fixed priority, round-robin and priority-with-aging as baselines.
5. Vary which sensor has the higher priority.

The headline result is the before/after comparison in experiments 1 and 2.

9. Workflow and Milestones

Phase	Work	Output / exit criterion
0. Setup	Vivado project, board files for Zedboard, repo structure, naming and interface conventions	Empty project builds; Zedboard PMOD pin map drafted
1. SPI foundation	spi_master and loopback testbench	ALL TESTS PASSED (in progress)
2. ACL2 path	adxl362_ctrl, slave model, testbench; then on hardware read DEVID 0xAD and live XYZ	id_ok LED lit; 100+ samples read with no loss
3. Interrupt block	irq_sync_pending with polarity option, tested with narrow and wide pulses	No lost pulses in simulation
4. I2C and HYGRO path	i2c_master, hdc1080_ctrl, slave model; hardware read of IDs and one measurement	Manufacturer ID 0x5449 read back; temperature and humidity plausible
5. UART output	uart_tx, formatter, FIFO	Readable text in a PC terminal
6. OLED output	SSD1306 init, font ROM, text write	Static text then live values on the display
7. Fixed-priority arbiter	Integrate both sensors and both outputs, mux, mutual exclusion	Both streams appear intact under simultaneous interrupts
8. Aging and configuration	Aging counters, parameters, config registers, enable bit	Aging on/off switch works in simulation
9. Performance counters and stress bench	perf_counters, stress testbench, synthetic interrupt generator	Metrics printed from simulation
10. Hardware integration	Synthesis, implementation, timing closure at 100 MHz, bitstream, ILA debug	Full demo on the board, WNS non-negative

Phase	Work	Output / exit criterion
11. Experiments	Run the experiment set in Section 8, collect tables and plots	Data for the report
12. Report and demo	Write-up, block diagrams, waveforms, results, demo rehearsal	Final submission

Build and test each block in isolation before integrating. The most common source of lost time is debugging three buses at once on hardware, so insist that every module passes its own testbench first.

10. Deliverables Checklist

- Verilog source for all modules listed in Section 4.1
- Self-checking testbenches with behavioral sensor models and the stress testbench
- Zedboard XDC with PL PMOD pin mappings and clock constraint
- Synthesis and implementation reports (utilization, timing summary)
- Measured latency and fairness tables, with and without aging
- Waveform screenshots of key behaviors (simultaneous interrupt, aging promotion, non-preemptive service)
- Live hardware demo: OLED and UART showing both sensors
- Project report

11. Risks and Mitigations

Risk	Why it happens	Mitigation
SPI returns garbage	Wrong mode or wiring; no ACK in SPI to tell you	Read DEVID_AD (0xAD) first; check CPOL/CPHA and pins
I2C never ACKs	Wrong address, SDA not tri-stated correctly, missing pull-ups	Use an open-drain IOBUF style tri-state; HYGRO board has pull-ups; test with the ID registers

Risk	Why it happens	Mitigation
DRDYn polarity bug	Active low versus ACL2 active high	Parameterize polarity in irq_sync_pending
Lost interrupts	Pulse arrives mid-transfer	Sticky pending flag plus a documented policy for repeat events
Rare contention on hardware	Sensors are slow and independent	Higher data rates and the synthetic interrupt generator
OLED stays blank	Power sequencing (VDDC, VBATC) and reset timing are strict	Follow the Digilent reference sequence exactly; add delays as parameters
Timing failures	Large combinational paths, wide counters	Pipeline, use clock enables, check WNS early
Simulation takes forever	Real millisecond waits	Parameterize all waits so testbenches use tiny values
Metastability	Async inputs	2-flop synchronizers on every external input including MISO and SDA

12. Suggested Report Outline

1. Introduction and motivation (polling versus interrupts, shared resources)
2. Background (SPI, I2C, UART, arbitration, starvation, aging)
3. System architecture and module descriptions
4. Arbitration and aging algorithm, with parameters
5. Verification strategy and testbenches
6. Experimental setup and metrics
7. Results: latency, fairness, aging on versus off, parameter sweeps
8. Hardware demo and resource utilization
9. Discussion: trade-offs, limitations, possible extensions
10. Conclusion

13. Possible Extensions (if time permits)

- Runtime-configurable priorities through UART receive commands
- A third requester to show scaling
- Weighted fair queuing as an alternative to aging
- Moving the controller under software supervision through the Zynq PS

14. Current Status

- Done: project understanding, module plan, spi_master.v, loopback testbench
- Next: run the loopback test to ALL TESTS PASSED, then adxl362_ctrl.v with a slave model, then the interrupt synchronizer and pending block